sentinel-oo: user manual

Anil Wipat, Newcastle University

11 November 2026

Contents

sentinel-oo: user manual **2**
- What you will end up with . . . 2
- Before you start . . . 3

Part 1: a tutorial **3**
- Step 1: look at what you have been given . . . 3
- Step 2: ask the question, then write one requirement . . . 3
- Step 3: the trap, which you should walk into on purpose . . . 4
- Step 4: tell the contract what it is for . . . 4
- Step 5: build up the requirements about your own entities . . . 4
- Step 6: a flag rather than a refusal . . . 5
- Step 7: a value a model may be asked to recover . . . 6
- Step 8: the second lesson . . . 6
- Step 9: read your contract back in plain words . . . 7
- Step 10: deploy it . . . 7

Part 2: the contract language **7**

Part 3: deploying **8**

Part 4: when something goes wrong **8**

Part 5: before you rely on it **9**

Writing a contract **10**
- What a contract is . . . 10
- The one question to ask about every requirement . . . 10
- The shape of the file . . . 10
- `expects`: what your contract is for . . . 10
- Every field of a requirement . . . 11
- `applies_to`: where it looks . . . 12
- `property` or `named_value`: what it reads . . . 12
- `must`: the four verbs . . . 12
- `severity`: how hard it bites . . . 14
- `one_of`: the only values that mean anything . . . 14
- `repairable`: letting a model help . . . 14
- What happens when you get it wrong . . . 15
- What the language cannot say yet . . . 15
- Checking your contract before you deploy . . . 15

Deploying your own instance **17**
- One command . . . 17
- What it does, in order . . . 17

What you see when it works . . . 17
The options . . . 18
Using it once it is running . . . 18
Two instances at once . . . 18
For a coordinator agent rather than a pipeline . . . 19
A model is optional . . . 19
What your deployment folder needs . . . 19
If you prefer Docker Compose . . . 19
Stopping . . . 19

sentinel-oo: user manual

How to put a checking service at one of your own handovers. You will write contract files and run one command. You will not write any code.

This manual is in three files. This one is the front door and the tutorial. The other two are reference:

File	What it is for
This file	Start here. A tutorial that builds a contract from nothing
WRITING_A_CONTRACT.md	The contract language, field by field
DEPLOYING.md	The deployment program, option by option

If you want to know how the system is built rather than how to use it, read the system manual.

What you will end up with

A service running in a container that you send a handover to and get a verdict back from, plus the handover itself with a record of what was checked written inside it.

A **handover** is the moment one phase of work passes its output to the next. A **crate** is how that output travels: an RO-Crate, which is a folder of data files with a JSON file alongside them describing what each file is and how the things in it relate to one another.

The three verdicts

Verdict	What it means
`ACCEPTED`	Nothing was found
`ACCEPTED_WITH_FLAGS`	Nothing stops the handover, but something was recorded for somebody to look at
`REFUSED`	At least one requirement with severity `refuse` was violated

What makes your instance yours

A folder of contract files, mounted into an image nobody modified. That is the whole of the tailoring.

Before you start

You need Docker, and a crate to try it on. If you have no crate of your own yet, use the worked example that ships with the product, which is what this tutorial does.

Everything below is run from the `sentinel-oo` folder.

Part 1: a tutorial

You can run every command here. The tutorial builds a contract one requirement at a time against the worked example that ships with the product, and the output shown is the output you will get.

The worked example is a coating laboratory. Recipes are blended, applied to metal panels held in numbered slots on a rack, dried, and their film thickness measured in micrometres. It is deliberately not biology, so that nothing in the product looks as if it belongs to somebody's use case.

Step 1: look at what you have been given

```
PYTHONPATH=src python -m sentinel_oo examples/coating-test/valid --out /tmp/out
```

That crate has 28 entities. The ones that matter for this tutorial are:

Entity type	What it is	How many
`CoatingRecipe`	A recipe to be applied	3
`RackSlot`	A numbered position on the rack	3
`IntendedRackLayout`	The rack itself	1
`ThicknessReading`	A measurement	3
`DryingStep`	The process step that dried the panels	1

Step 2: ask the question, then write one requirement

The question to ask about every requirement is: what does the receiving phase do, and what goes wrong if this is missing?

If a requirement has no answer to that, it is a preference rather than a requirement. Preferences accumulate until a contract is too irritating to use, and then people route around it, and then it protects nothing.

Here is one with an answer. If a slot has no position, nobody can find it on the physical rack.

Make a folder and put this in it as `my_handover.yaml`:

```
contract: my_handover
version: 0.1.0

requires:
  - name: slot_has_a_position
    applies_to: RackSlot
    property: name
    must: be recorded
    severity: refuse
    message: >-
      This slot has no position, so nobody can find it on the physical rack.
```

Run it:

```
SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/coating-test/valid \
  --contract my_handover

  28 entities in, 41 out
  verdict                                ACCEPTED
```

Every slot has a position, so it passes.

Step 3: the trap, which you should walk into on purpose

Send it a crate from a completely different domain. One ships with the product for exactly this purpose: a library catalogue, which has no slots, no recipes and no drying step in it at all.

```
SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/from-another-phase \
  --contract my_handover

  verdict                                ACCEPTED
```

It accepted a crate it knows nothing about. Your requirement is about `RackSlot`, that crate contains no `RackSlot`, so the requirement matched nothing and passed.

The requirement is behaving correctly. A rule about a type should say nothing about a crate with none of that type. The problem is the verdict drawn from its silence.

This is the worst thing the service can do, and it is worth understanding before you write anything else. Refusing something good is visible and sends somebody to look. Blessing something unchecked is silent and looks like success.

Step 4: tell the contract what it is for

Add an `expects` block listing the entity types your contract is written for:

```
contract: my_handover
version: 0.2.0

expects:
  - RackSlot

requires:
  - name: slot_has_a_position
    ...
```

Now run both crates again:

```
  the coating crate     ACCEPTED
  the other domain      REFUSED
                        REFUSE  nothing of type RackSlot is in this crate,
                                and the contract my_handover is written for
                                crates that contain it
```

Write this block in every contract you ever write.

Step 5: build up the requirements about your own entities

Add the rest. Notice that references come in pairs: one requirement that the reference is recorded at all, and a second that it lands on something real. They are separate because one rule doing both would make one of them impossible to say.

```
expects:
  - RackSlot
  - CoatingRecipe
  - DryingStep

requires:
  - name: slot_has_a_position
    applies_to: RackSlot
    property: name
    must: be recorded
    severity: refuse
    message: >-
      This slot has no position, so nobody can find it on the physical rack.

  - name: one_slot_per_position
    applies_to: RackSlot
    property: name
    must: be unique
    severity: refuse
    message: >-
      Two slots claim the same position, so what is in that position is
      undefined.

  - name: slot_names_a_recipe
    applies_to: RackSlot
    property: mainEntity
    must: be recorded
    severity: refuse
    message: >-
      This slot does not say which recipe goes on it.

  - name: slot_names_a_real_recipe
    applies_to: RackSlot
    property: mainEntity
    must: point at something in the crate
    of_type: CoatingRecipe
    severity: refuse
    message: >-
      This slot names something that is not a recipe in this crate.
```

Step 6: a flag rather than a refusal

Not everything should stop a handover. The coating line cannot run below 5 micrometres or above 500 micrometres, where a micrometre is one thousandth of a millimetre. But a recipe outside that range may be intended for a different line, which is a judgment for a person rather than a reason to stop.

```
  - name: recipe_thickness_is_in_range
    applies_to: CoatingRecipe
    named_value: film thickness
    must: be a number between
    between: [5, 500]
    severity: flag
    message: >-
      This recipe's film thickness, in micrometres, is outside what the
      coating line can run, which is 5 to 500 micrometres.
```

`severity: refuse` stops the handover. `severity: flag` records the finding and lets it through. There is nothing in between, on purpose: a severity nobody can act on is a severity nobody reads.

State the units in every message. The contract has no field for them, and a number with no unit in a finding is an invitation to compare a thickness against a temperature.

Step 7: a value a model may be asked to recover

```
- name: drying_atmosphere_is_recorded
  applies_to: DryingStep
  named_value: drying atmosphere
  must: be recorded
  repairable: true
  severity: refuse
  message: >-
    The drying step does not say what atmosphere it ran in, so these
    thickness readings cannot be compared with each other.
```

`repairable: true` means a model may be asked one question when this fails: is the value already written down somewhere in the record, including inside free text? Everything it answers is checked by your rules before it counts, and by default no model is called at all.

Now run all five example crates:

```
valid                          ACCEPTED
invalid/missing-conformsto     ACCEPTED
invalid/dangling-reference     ACCEPTED
invalid/missing-atmosphere     REFUSED
invalid/atmosphere-in-prose    REFUSED
```

Step 8: the second lesson

Two broken crates were accepted. **A contract catches only what it asks about**, and yours asks nothing about the packaging: whether the crate says which profile it follows, or whether it lists files it never describes.

Add those:

```
- name: declares_a_profile
  applies_to: root
  property: conformsTo
  must: be recorded
  severity: refuse
  message: >-
    The crate does not say which profile it follows, so nothing can be
    agreed about what its contents mean.

- name: everything_the_crate_lists_is_present
  applies_to: root
  property: hasPart
  must: point at something in the crate
  severity: refuse
  message: >-
    The crate lists a file as part of itself that is described nowhere in
    it. Whoever receives this will look for it and not find it.
```

`applies_to: root` means the entity describing the crate as a whole.

```
valid                          ACCEPTED
invalid/missing-conformsto     REFUSED
invalid/dangling-reference     REFUSED
invalid/missing-atmosphere     REFUSED
invalid/atmosphere-in-prose    REFUSED
```

And a refusal reads like this:

```
  verdict                                REFUSED

  REFUSE  The crate lists a file as part of itself that is described
          nowhere in it. Whoever receives this will look for it and not
          find it.
         at ./
```

The sentence is yours. Write the message you would want to receive at four o'clock on a Friday.

Step 9: read your contract back in plain words

```
PYTHONPATH=src python -m sentinel_oo --show-contract my_handover

my_handover version 0.8.0

It is for crates containing all of: RackSlot, CoatingRecipe, DryingStep
A crate containing none of those is refused rather than accepted without
being checked.

8 requirement(s):

  declares_a_profile
    root must record conformsTo
    failing it refuses the handover
    The crate does not say which profile it follows, so nothing can be
    agreed about what its contents mean.
  ...
```

This is what you send to the team that produces the handover, so they can check themselves before they are checked. It is not YAML and they do not need to read YAML.

Step 10: deploy it

```
python deploy.py <your folder of contracts>
```

That validates your contracts and refuses before building anything, builds the generic image, starts an instance with your folder mounted read only, and then confirms the running instance is serving exactly what it validated.

```
curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/my_handover
```

Full details in DEPLOYING.md.

Part 2: the contract language

Every field, all four verbs, and the three things the language cannot say yet: WRITING_A_CONTRACT.md.

The four verbs are the whole vocabulary:

Verb	What it demands
`be recorded`	The value is present and not empty
`be unique`	No two entities of this type share it
`point at something in the crate`	The reference resolves to something described here
`be a number between`	It is a number, within a range

You cannot add a fifth by writing a contract. If you need one, you will get a message naming the four that exist and saying where a fifth would be added, and you are blocked until somebody adds it to the product.

Part 3: deploying

DEPLOYING.md covers the deployment program, its options, running more than one instance, and the interface for a coordinating agent.

Part 4: when something goes wrong

What you see	What it means
`does not understand ['propery']`	A misspelt field. The message lists every field a requirement may have
`says 'be greater than', which is not something a requirement can demand`	A verb that does not exist. The message lists the four that do
`has no message`	Every requirement needs a sentence a person can read
`has severity 'warn'`	It is `refuse` or `flag`, and nothing else
`says 'be a number between' but does not say between`	That verb needs a `between: [low, high]`
`must say either property, for something written on the entity itself, or named_value`	A requirement reads one or the other, never both and never neither
`two files define the contract 'x'`	Two files in your folder declare the same contract name. The name inside the file is what matters, not the file name
`no contract called 'x'. This service knows: none at all.`	Nothing was mounted at `/contracts`, or the wrong folder was
`the running instance is not serving the contracts that were validated`	Usually the wrong folder being mounted. The message names what it expected and what it found

Everything is refused loudly rather than ignored. A contract containing something the reader does not understand is rejected whole, because a silently ignored requirement is a requirement you believe is protecting you.

It accepted something I expected it to refuse

Three usual causes:

Your contract does not ask about it. A contract catches only what it asks about. Step 8 of the tutorial is this mistake.

The type name does not match. Types are compared as plain text, and nothing knows that schema.org considers one type a kind of another. If your crate says `CoatingRecipe` and your contract says `Recipe`, the requirement matches nothing and passes.

You have no `expects` block. Run `--show-contract`. If it warns that the contract does not say what it is for, that is the cause.

Part 5: before you rely on it

A checklist. Each line is a mistake somebody has actually made.

- ☐ Every contract has an `expects` block listing the types it is for
- ☐ Every reference has a pair: one requirement that it is recorded, one that it lands on something real
- ☐ Every message is a sentence you would want to receive, and states units for every number
- ☐ Every requirement answers “what does the next phase do, and what goes wrong if this is missing?”
- ☐ You have run it against a crate you know is good **and** a copy you have deliberately broken
- ☐ `--show-contract` reads correctly to somebody who has never seen YAML
- ☐ You have sent that plain-words version to the team producing the handover
- ☐ `be unique` is only used where duplication really is wrong in your laboratory, not where it is how a replicate or a control is included

A contract that accepts everything is the same as no contract. The only way to know yours is doing something is to watch it refuse something.

Writing a contract

This is the whole of what you need in order to tailor `sentinel-oo` to your own handover. You will not write any code.

Read the worked example alongside this. It is 2 contract files and 5 crates, and it is short on purpose.

What a contract is

A handover is the moment one phase of work passes its output to the next. A contract is a file saying what must be true of that output before the next phase can act on it.

The service reads your contract, checks a crate against it, and returns one of three verdicts: ACCEPTED, ACCEPTED_WITH_FLAGS, or REFUSED. The middle one means nothing stopped the handover but something was recorded for a person or a later step to look at. A crate here means an RO-Crate: a folder of data files with a JSON file alongside them describing what each file is and how the things in it relate to each other.

What a contract is not. It is not a schema, because it does not say what shape the data must have. It is not a validator, because a third party validator already checks that the packaging is well formed. It is not a set of instructions to a model: there is no field for that, deliberately, and the reader refuses a file that invents one.

The one question to ask about every requirement

What does the receiving phase do, and what goes wrong if this is missing?

If a requirement has no answer to that, it is a preference rather than a requirement. Preferences accumulate until a contract is too irritating to use, and then people route around it, and then it protects nothing.

The shape of the file

```
contract: mixing_to_coating
version: 1.1.0

description: >
  What Mixing must hand to Coating, and why each item matters.

expects:
  - CoatingRecipe
  - RackSlot

requires:
  - name: declares_a_profile
    applies_to: root
    property: conformsTo
    must: be recorded
    severity: refuse
    message: >-
      The crate does not say which profile it follows, so nothing can be
      agreed about what its contents mean.
```

Five top level keys and nothing else: `contract`, `version`, `description`, `expects`, `requires`. The name in `contract` is how callers ask for it, as in `POST /check/mixing_to_coating`. The file name does not matter; this name does.

`expects`: what your contract is for

Write this block. A contract without one accepts crates it has not checked.

`expects` lists the entity types your contract is written for. If any of them is absent from the crate, the handover is refused before a single requirement is applied, and the message names the type that is missing.

```
expects:
  - CoatingRecipe
  - RackSlot
  - IntendedRackLayout
  - ThicknessReading
  - DryingStep
```

Why it is not optional in practice

A requirement about a type says nothing about a crate containing none of that type, and it should not, because presence and soundness are separate things. Each requirement is behaving correctly. The problem is what happens when you add them up.

This was measured on 11 November 2026. A crate from an unrelated domain was posted to a contract of 18 requirements. The result was ACCEPTED with 0 findings:

Of the 18 requirements	Count
Looked at something in that crate	6
Matched no entity at all, and passed	12

The 12 were every requirement naming a type the crate did not contain. So the service said ACCEPTED on a crate it had not meaningfully read.

That is the worst thing this service can do. Refusing something good is visible and sends somebody to look. Blessing something unchecked is silent and looks like success. A handover from the wrong phase, or a crate in which one type name has been changed or misspelt, would come back accepted.

Why you choose the list rather than the service inferring it

The service could gather every type your requirements mention and demand all of them. It does not, because a contract may legitimately carry requirements about entity types that are optional, and only you know which of yours must be present. Inferring it would turn optional things into mandatory ones without asking.

So list the types whose absence means this crate is not the handover your contract is about.

Every field of a requirement

Field	Required?	What it does
`name`	Yes	How this requirement is referred to in a finding. Use words, not numbers
`applies_to`	Yes	Which entities it looks at
`property`	One of these two	A property written directly on the entity
`named_value`	One of these two	A value hanging off the entity as a separate entity
`must`	Yes	What it demands. One of exactly four things
`severity`	Yes	`refuse` or `flag`
`message`	Yes	The sentence a person reads when it is violated

Field	Required?	What it does
`one_of`	No	The only values that mean anything
`of_type`	No	Narrows a reference to one kind of thing
`between`	Only with the range verb	Two numbers, lowest first
`repairable`	No	Whether a model may be asked to recover the value

A requirement must say either `property` or `named_value`, never both and never neither.

`applies_to`: where it looks

Three possibilities.

`root` is the entity describing the crate as a whole. Use it for things true of the package: its name, its licence, which profile it follows, the list of files it claims to contain.

`descriptor` is the entity describing the metadata file itself. You will rarely want it.

Anything else is read as a type, so `applies_to: CoatingRecipe` means every entity in the crate that says it is a `CoatingRecipe`. You may write `type:CoatingRecipe` if you want to be explicit.

A type here means either the entity's `@type` or its `additionalType`. The second matters in practice: RO-Crate's vocabulary has no term for a coating recipe, so the worked example types those entities as `CreativeWork` with `additionalType: CoatingRecipe`, and a requirement can then name them.

A limitation worth knowing before you rely on it. Types are compared as plain text. Nothing in the service knows that schema.org considers a `Hotel` to be a kind of `LodgingBusiness`, so a requirement asking for one will not accept the other. If your crate uses a specific type and your contract names a general one, the requirement will match nothing and pass silently.

`property` or `named_value`: what it reads

`property` reads something written on the entity itself:

```
applies_to: RackSlot
property: name
```

`named_value` reads a value hanging off the entity as a separate entity of its own:

```
applies_to: CoatingRecipe
named_value: film thickness
```

Both exist because crates following the Process Run Crate profile record most numbers and strings the second way. The film thickness is not written on the recipe. It is a separate entity carrying the name `film thickness` and the value `42`, which the recipe points at.

You do not have to say which property carries the reference. Every reference the entity makes is followed, and only entities of type `PropertyValue` are read as values. So a step that points at a file, a tool and a setting will give up only the setting, and a file that happens to have a name is not mistaken for a value.

`must`: the four verbs

This is the whole vocabulary. It is deliberately short, and it grows one entry at a time, because a language that can say anything can also say things nobody can check.

You cannot add a verb by writing a contract. If your requirement needs one these four cannot express, you will get a message naming the four that exist, and you are blocked until a fifth is added to the product. That

means editing `MUSTS` in `src/sentinel_oo/contract.py`, adding a branch to `_apply` in `src/sentinel_oo/rules.py`, and writing tests for both. It is the one case where tailoring is not enough, and it is a change somebody has to agree to rather than something you can do in your own folder.

`be recorded`

The value must be present and not empty.

```
- name: slot_has_a_position
  applies_to: RackSlot
  property: name
  must: be recorded
  severity: refuse
  message: >-
    This slot has no position, so nobody can find it on the physical rack.
```

This is the only verb that fires when the value is absent. The others have nothing to say about something that is not there, because "it must be present" and "it must be sound" are two separate requirements. A rule doing both would make one of them impossible to say.

So presence and soundness come in pairs. Write both.

`be unique`

No two entities of this type may share this value.

```
- name: one_slot_per_position
  applies_to: RackSlot
  property: name
  must: be unique
  severity: refuse
  message: >-
    Two slots claim the same position on the rack, so what is physically in
    that position is undefined.
```

Think about whether duplication is actually wrong in your laboratory. Two slots holding the same recipe is a normal thing to do elsewhere: it is how a replicate or a control is included. A check that fires on correct practice is worse than no check. If replicates are normal for you, use `severity: flag` or leave the requirement out.

`point at something in the crate`

The reference must resolve to an entity the crate describes.

```
- name: slot_names_a_real_recipe
  applies_to: RackSlot
  property: mainEntity
  must: point at something in the crate
  of_type: CoatingRecipe
  severity: refuse
  message: >-
    This slot names something that is not a recipe in this crate, so
    whatever is on it cannot be identified.
```

With `of_type`, the thing it lands on must also be of that type. Without it, anything in the crate will do.

A reference that points outside the crate, to a URL somewhere, counts as not resolving. That is the point: a crate that refers to things it does not carry or describe cannot be acted on by somebody who only has the crate.

`be a number between`

The value must be a number, and within the range.

```
- name: recipe_thickness_is_in_range
  applies_to: CoatingRecipe
  named_value: film thickness
  must: be a number between
  between: [5, 500]
  severity: flag
  message: >-
    This recipe's specified film thickness, in micrometres, is outside what
    the coating line can run, which is 5 to 500 micrometres.
```

`between` takes two numbers, lowest first, and the range includes both ends.

State the units in the message every time. The contract has no field for units, and a number with no unit in a finding is an invitation to compare a thickness against a temperature.

`severity`: how hard it bites

Two values and nothing in between.

`refuse` stops the handover. The verdict becomes REFUSED and whatever comes next should not run.

`flag` records the finding and lets the handover through. The verdict stays ACCEPTED and the finding is written into the returned crate, where something downstream can act on it or a person can read it.

There is no middle value on purpose. A severity nobody can act on is a severity nobody reads.

Use `flag` when the answer is a judgment for a person, and `refuse` when proceeding would waste money or produce a record that is wrong.

`one_of`: the only values that mean anything

```
- name: drying_atmosphere_is_dry
  applies_to: DryingStep
  named_value: drying atmosphere
  must: be recorded
  one_of: ["dry"]
  severity: refuse
  message: >-
    This contract is written for panels dried in a dry atmosphere. Panels
    dried any other way cure differently, and need a contract that says
    what to do about them.
```

A known awkwardness, stated so it does not surprise you. `one_of` hangs off `be recorded`, so when the value is absent both the presence requirement and this one fire, and the report says two things rather than one. Both are true, but it is noise. There is no way to say “if present, must be one of these” yet.

`repairable`: letting a model help

```
repairable: true
```

Marking a requirement repairable means a model may be asked one question when that requirement fails: **is this value already written down somewhere in the record, including inside free text?**

That is the whole of what a model does here. It is the one job rules cannot do, because a rule can only read a value recorded as a value, and a value sitting inside a sentence is invisible to it.

Everything a model proposes is then checked by your rules before it counts, and it is kept only if every requirement reading that value now passes and nothing new is broken anywhere. The returned crate records that the value was recovered from text rather than stated, and which engine proposed it.

Repairable is not the same as guessable. Nothing may be invented. The value either is in the record or it is not, and answering that it is not there is a correct and expected answer.

What this was measured to do. On 10 November 2026 two models running locally, each with 14 billion parameters, were asked to do exactly this job on a real crate. Neither could. Every outcome was a refusal with nothing written, which is the system behaving correctly. A model being wrong, slow, absent or confidently mistaken changes no verdict. Mark a requirement repairable because it would be useful if it worked, not because you are relying on it.

By default no model is called at all. The repair budget defaults to 0, so `repairable: true` does nothing until a caller asks for repair.

What happens when you get it wrong

The reader refuses the whole contract and names the fault. It does not ignore what it does not understand, because a silently ignored requirement is a requirement you believe is protecting you.

```
trial.yaml: requirement 1 (r) says 'be greater than', which is not something
a requirement can demand. It may say: ['be a number between', 'be recorded',
'be unique', 'point at something in the crate'].

trial.yaml: requirement 1 does not understand ['propery']. A requirement may
say ['applies_to', 'between', 'message', 'must', 'name', 'named_value',
'of_type', 'one_of', 'property', 'repairable', 'severity'] and nothing else.
In particular there is no field for instructing a model.

trial.yaml: requirement 1 (r) has no message. A violation nobody can read is
not worth raising.

trial.yaml: requirement 1 (r) has severity 'warn'. It is ['flag', 'refuse']:
either the handover stops or it is mentioned, and there is nothing in between
that anybody can act on.
```

What the language cannot say yet

Three gaps, recorded rather than worked around. If you need one of these, you are blocked until a verb is added to the product, and that is a change to the code rather than to your file.

It cannot compare one value against another. You cannot say that a recipe's thickness must be about three times its primer's thickness, or that one length must be consistent with another. Every requirement reads one value and judges it on its own.

There are no conditional requirements. You cannot say that if the drying atmosphere was humid then a correction factor must also be recorded. Each requirement fires or does not fire regardless of anything else in the crate.

There is no way to say "if present, must be one of these". `one_of` hangs off `be recorded`, so a missing value triggers both requirements.

Checking your contract before you deploy

The service prints a contract back in plain words, which is what a producing team reads to check themselves before they are checked:

```
python -m sentinel_oo --show-contract mixing_to_coating
```

Then run it against a crate you know is good, and against a copy you have deliberately broken. A contract that accepts everything is the same as no contract, and the only way to know yours is doing something is to watch it refuse something.

See DEPLOYING.md for getting it running.

Deploying your own instance

You need a folder of contract files. That is the whole of what makes an instance yours, and you will not write any code.

See WRITING_A_CONTRACT.md for the contract language.

One command

```
python deploy.py <your folder of contracts>
```

For example, using the worked example that ships with the product:

```
python deploy.py examples/contracts
```

The deployment is a program rather than a page of instructions on purpose. Nothing is tailored by hand and no person follows steps.

What it does, in order

It stops at the first thing that fails, and the exit code is 1 when it stops.

1. It reads every contract in your folder and refuses the deployment if any of them is malformed, naming the file, the requirement and what a requirement may say.

This happens before anything is built or started. A deployment that starts with a broken contract discovers the problem at first use, which means discovering it in front of whoever sent the handover. Finding out here costs nobody anything.

It also refuses a folder that holds no contract files at all, because an instance with none would refuse every handover it was given.

2. It builds the generic image from the product folder. The product's own test suite, 200 tests, runs inside that build, and the image cannot be produced if any of them fails.

Pass `--no-build` to use an image that already exists.

3. It starts an instance with your contracts folder mounted read only at `/contracts`. The image is not modified and holds no contracts of its own.

Read only is deliberate: a service that checks handovers has no business editing the contract it checks against.

4. It checks the running instance is serving what was validated, comparing the contract names, their versions, and a checksum over the files.

That catches two things. The wrong folder being mounted, which is the common mistake. And a contract edited between step 1 and step 3, which is a real window because the build takes about a minute.

What you see when it works

```
sentinel-oo: deploying a tailored instance

1. Reading contracts from examples/contracts
   coating_to_inspection        0.2.0      2 requirements
   mixing_to_coating            1.1.0     19 requirements
   checksum over all of them: sha256:dd5dd91a...

2. Building the generic image
   sentinel-oo:dev built
   the product's test suite ran inside the build and passed

3. Starting the instance
```

```
   container sentinel-examples started from sentinel-oo:dev
   contracts mounted read only from examples/contracts

4. Checking the instance serves what was validated
   answering on http://127.0.0.1:8080
   image dev, commit c0525b0, model engine none
   2 contract(s) served, checksum matches what was validated

Deployed.
```

The options

Option	What it does
`--check-only`	Validate the contracts and stop. Nothing is built or started
`--no-build`	Use the existing image instead of building
`--crates <folder>`	Mount a folder of crates read only at `/crates`, for trying it
`--port <n>`	Which port to serve on. The default is 8080
`--name <n>`	The container name. The default is `sentinel-<your folder's parent>`
`--image <n>`	The image to build and run. The default is `sentinel-oo:dev`
`--stop`	Stop and remove the instance

Using it once it is running

```
curl -s http://127.0.0.1:8080/contracts

curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/<your contract name>
```

The reply carries the verdict, the findings, and the crate with the service's record of what it checked written inside it.

Method and path	What it does
`POST /check/{contract}`	The whole job. A crate in, a checked crate out
`GET /contracts`	Which contracts this instance holds, with versions and a checksum
`GET /contracts/{name}`	One contract written out in plain words
`GET /runs`	The runs this instance is still holding
`GET /health`	The build it is running and which model, if any
`GET /openapi.json`	A description of the service, generated from its route table

Two instances at once

The image is generic, so one image serves any number of deployments. Give each one its own port and name:

```
python deploy.py <your own contracts> --port 8080
python deploy.py examples/contracts --port 8081 --name sentinel-example
```

That was run on 11 November 2026 and both instances served at the same time from one unmodified image, each holding only its own contracts and each refusing a contract name the other one knew.

For a coordinator agent rather than a pipeline

The same image speaks the Model Context Protocol on standard input and output instead of HTTP. An agent starts it when it wants it:

```
docker run -i --rm \
  -v "$PWD/<your contracts>:/contracts:ro" \
  sentinel-oo:dev --agent-interface
```

It offers four tools: `check_handover`, `list_handovers`, `read_contract` and `get_run_graph`. It never returns the crate, only the verdict, the findings and the path the crate was written to, because a crate is thousands of lines of JSON and putting that in an agent's context costs money on every call and invites the agent to start editing laboratory records.

A test fails if the two interfaces ever give different verdicts.

Not yet tested against a real Model Context Protocol client. It is tested against the protocol as written down.

A model is optional

With no model configured the service refuses and accepts exactly as it does with one, except that it cannot recover a value that is present in a crate only inside free text. The repair budget defaults to 0, so by default no model is called at all.

Two local models with 14 billion parameters each were measured on this on 10 November 2026 and neither could do the job. That measurement is in the product README. A model being wrong, slow, absent or confidently mistaken changes no verdict and writes nothing into any record.

What your deployment folder needs

Only the contracts. Everything else is optional:

```
your-deployment/
  contracts/
    your_handover.yaml        the whole of the tailoring
  crates/                     optional, for trying it
  README.md                   optional, what this deployment is for
```

There is a worked one in `examples/`.

If you prefer Docker Compose

`deploy.py` uses `docker run` directly, so a deployment needs no compose file at all. A hand written Compose file can do the same thing, but without the validation in steps 1 and 4, so a broken contract will not be caught until first use.

Stopping

```
python deploy.py <your folder of contracts> --stop
```